# SIMATS ENGINEERING
## ASSESSMENT TOOL 1

**COURSE NAME: Artificial Intelligence** **COURSE CODE: CSA17**

---

**COURSE OUTCOME COVERED**

**CO5:** *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.* (BTL 5)

*Assessment Tool Weightage: 50%*

---

**QUESTIONS: 5** **Duration : 1 Hour**
**Total Marks:50**

Annexure A

## Design Task

---

***Code of Conduct:***

*I Arava Raja (192365033) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Arava Raja***
***Signature of the student***

Annexure A

# Design Task

1. **Hybrid AI Recommendation Engine Design**
   Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

2. **Smart Disaster Detection System**
   Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

3. **AI-Based Fraud Detection Framework**
   Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

4. **Autonomous Supply Chain Optimization System**
   Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

5. **Personalized Learning Analytics Platform**
   Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

## RUBRICS FOR CALCULATION

| Evaluation Criteria | Problem Understanding & Requirement Analysis | System Architecture Design | Use of PySpark & RDD Operations | Intelligent Agent Integration | Scalability & Distributed Computing Design | Data Flow & Processing Pipeline | Innovation & Creativity | Clarity, Presentation & Documentation |
|---|---|---|---|---|---|---|---|---|
| **Weightage** | 10% | 20% | 15% | 15% | 10% | 10% | 10% | 10% |

| Criteria | Excellent (5) | Good (4) | Average (3) | Poor (2) |
|---|---|---|---|---|
| Problem Understanding & Requirement Analysis | Clearly defines problem, objectives, and constraints | Mostly clear with minor gaps | Basic understanding | Unclear or incorrect |
| System Architecture Design | Complete, scalable, well-structured architecture with diagrams | Good design with minor issues | Basic design, lacks detail | Poor/incomplete |
| Use of PySpark & RDD Operations | Efficient and correct use of RDD transformations/actions | Mostly correct usage | Limited or partial use | Incorrect/missing |
| Intelligent Agent Integration | Strong integration with clear agent roles and logic | Good integration | Basic integration | Poor/no integration |
| Scalability & Distributed Computing Design | Clearly addresses scalability, fault tolerance, parallelism | Covers most aspects | Limited consideration | Not addressed |
| Data Flow & Processing Pipeline | Clear, logical, and well-defined data flow | Mostly clear | Some gaps | Poorly defined |

| | | | | |
|---|---|---|---|---|
| Innovation & Creativity | Highly innovative, realistic, and impactful solution | Some innovation | Basic idea | No originality |
| Clarity, Presentation & Documentation | Very clear, neat diagrams, well-organized report | Mostly clear | Somewhat organized | Poor presentation |

## 1. Hybrid AI Recommendation Engine Design

### Introduction

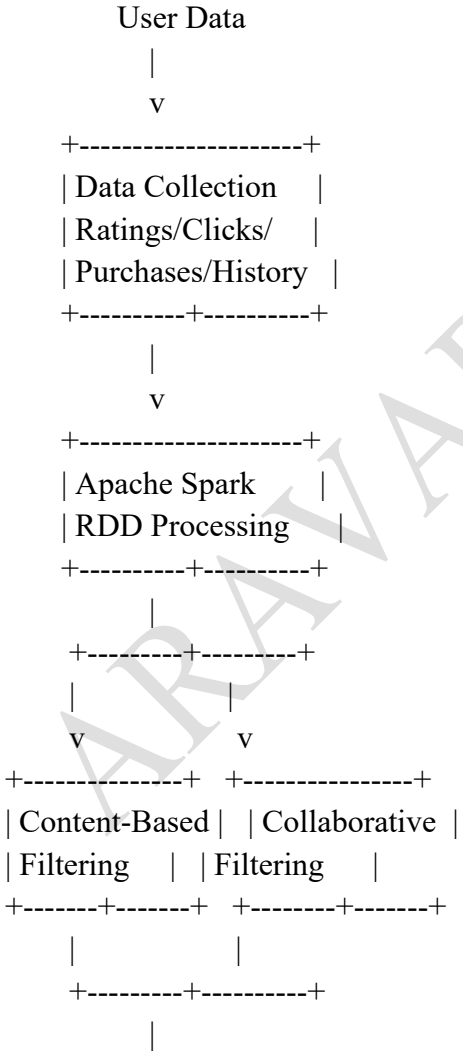
A Hybrid AI Recommendation Engine is an intelligent recommendation system that combines **content-based filtering** and **collaborative filtering** to provide personalized recommendations. The system uses **PySpark RDDs** to process large-scale user-item interaction data in a distributed environment.

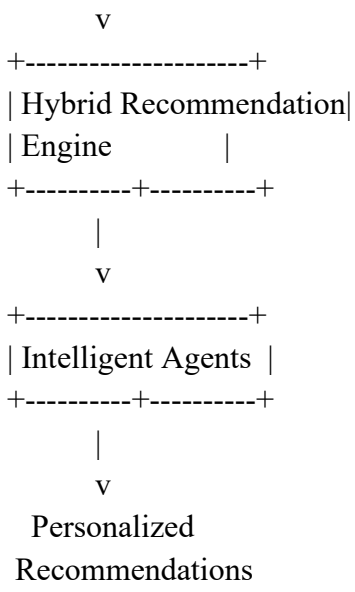
The system analyzes user preferences, previous purchases, ratings, browsing history, and item characteristics. Intelligent agents continuously monitor user behavior and feedback to update recommendations in real time.

### Objectives

- Provide personalized recommendations.
- Process millions of user-item interactions efficiently.
- Combine content-based and collaborative filtering.
- Adapt recommendations based on real-time feedback.
- Improve recommendation accuracy.
- Support thousands or millions of users.

### System Architecture






**Content-Based Filtering**

Content-based filtering recommends items similar to items previously liked by the user.

For example, if a user frequently watches action movies, the system recommends other action movies.

Features may include:

- Category
- Genre
- Price
- Brand
- Keywords
- Item description
- User interests

A feature vector is created for every item, and similarity between items can be calculated using **Cosine Similarity**.

**Collaborative Filtering**

Collaborative filtering uses the behavior of similar users.

For example:

User A and User B have purchased similar products. User A purchased Product X, but User B has not. Product X can therefore be recommended to User B.

The system can construct:

User → Item → Rating

and identify similar users or similar items.

**PySpark RDD Operations**

The interaction data can be represented as:

(userID, itemID, rating)

RDD transformations can include:

ratings.map(lambda x: (x[0], x[2]))

to extract user-rating information.

Other useful operations include:

- map()
- filter()
- flatMap()
- groupByKey()
- reduceByKey()
- join()
- mapValues()

Actions include:

- count()
- collect()
- take()
- reduce()

For example, reduceByKey() can calculate aggregate ratings efficiently across distributed partitions.

**Intelligent Agents**

Three major agents can be used:

**1. User Behavior Agent**

Monitors:

- Searches
- Clicks
- Purchases
- Ratings
- Watch time

**2. Recommendation Agent**

Combines content-based and collaborative results and generates recommendations.

**3. Feedback Agent**

Analyzes whether the user accepted, ignored, clicked, or purchased recommended items.

The feedback is then sent back to the recommendation engine.

**Real-Time Adaptation**

The system continuously updates user profiles.

For example:

User searches for laptops

↓

Agent detects new interest

↓

User preference updated

↓

Recommendation model updated

↓

New laptop recommendations

**Scalability**

PySpark distributes RDD partitions across multiple worker nodes. Therefore, millions of interactions can be processed in parallel.

Advantages include:

- Parallel processing
- Fault tolerance
- Horizontal scalability
- Faster computation
- Distributed memory processing

**Conclusion**

The proposed hybrid recommendation engine combines the advantages of content-based and collaborative filtering. PySpark provides scalable data processing, while intelligent agents continuously monitor user behavior and feedback. This produces accurate, adaptive, and personalized recommendations.

## 2. Smart Disaster Detection System

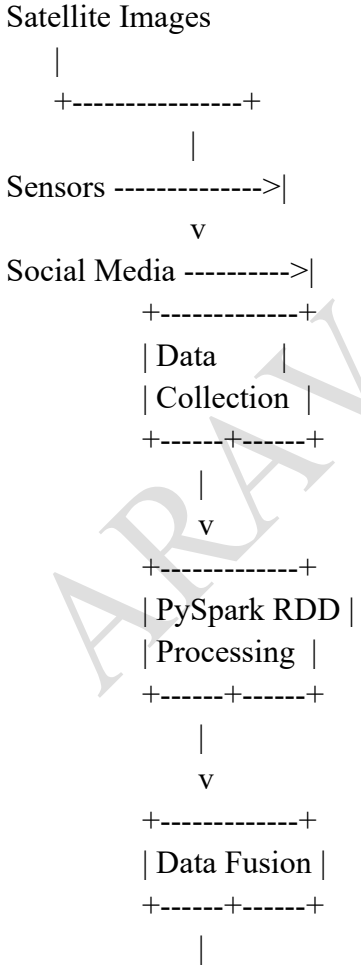
### Introduction

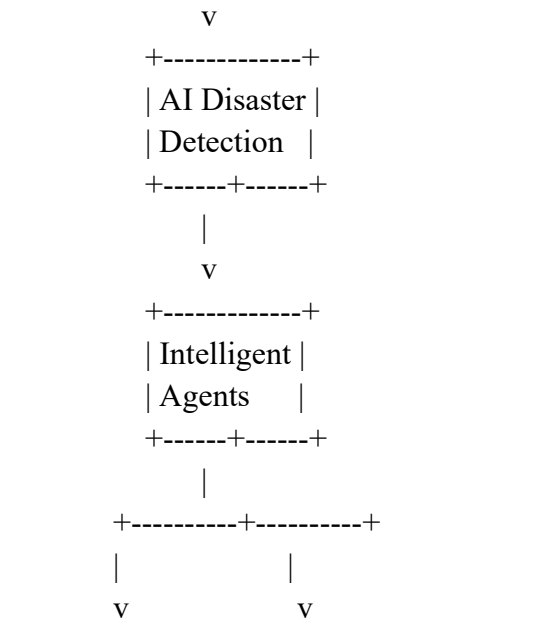
A Smart Disaster Detection System is a distributed AI system designed to detect natural disasters such as **floods and earthquakes** using multiple data sources including satellite images, sensors, and social media. The assessment specifically requires PySpark/RDD processing, data fusion, and intelligent-agent coordination.

### Objectives

- Detect disasters at an early stage.
- Process data from multiple sources.
- Identify abnormal environmental conditions.
- Generate early warnings.
- Reduce disaster response time.
- Coordinate emergency response.

### System Architecture






**Data Sources**

**Satellite Data**

Satellite images can provide information about:

- Flooded regions
- Cloud patterns
- Water-level changes
- Land movement

**Sensor Data**

Sensors can provide:

- Water levels
- Temperature
- Pressure
- Ground vibrations
- Rainfall

**Social Media**

Social media can provide real-time information such as:

- User reports
- Emergency messages
- Location information
- Disaster-related keywords

**PySpark RDD Processing**

Each data source can be converted into RDDs.

For example:

Sensor RDD

(location, water_level, timestamp)

Social Media RDD

(location, message, timestamp)

RDD transformations such as:

- map()
- filter()
- flatMap()
- reduceByKey()
- join()

can be used to process the data.

For example, sensor readings above a predefined threshold can be filtered:

```
sensorRDD.filter(lambda x: x[1] > threshold)
```

**Data Fusion**

Data fusion combines information from different sources.

For example:

High rainfall

+

High water level

+

Satellite detects flooded area

+

Social media reports flooding

↓

High-confidence flood detection

Using multiple sources reduces false alarms.

**Intelligent Agents**

**Sensor Monitoring Agent**

Continuously monitors sensor values.

**Satellite Analysis Agent**

Analyzes satellite information and detects environmental changes.

**Social Media Agent**

Identifies disaster-related messages and extracts locations.

**Decision Agent**

Combines information from all agents and determines disaster severity.

**Emergency Response Agent**

Sends alerts and recommends suitable response actions.

**Early Warning Process**

Data Collection

↓

RDD Processing

↓

Anomaly Detection

↓

Data Fusion

↓

Disaster Confirmation

↓

Severity Estimation

↓
Warning Generation
↓
Emergency Response

**Scalability**

PySpark allows data to be divided into multiple partitions and processed simultaneously. This is useful because disaster systems may receive thousands of sensor readings and social-media messages simultaneously.

**Conclusion**

The Smart Disaster Detection System provides a scalable and intelligent solution for early disaster detection. Combining multiple data sources with PySpark distributed processing and intelligent agents improves detection accuracy, reduces response time, and supports real-time emergency management.

**3. AI-Based Fraud Detection Framework**

**Introduction**

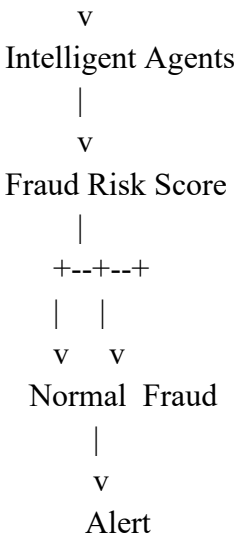
An AI-Based Fraud Detection Framework identifies suspicious financial transactions by processing large volumes of transaction data using PySpark RDDs. The assessment requires distributed anomaly detection and intelligent agents that collaborate to identify suspicious patterns and continuously improve detection accuracy.

**Objectives**

- Detect fraudulent transactions.
- Process large transaction datasets.
- Identify abnormal behavior.
- Generate real-time alerts.
- Reduce false positives.
- Continuously improve fraud detection.

**Architecture**

Financial Transactions
|
v
Data Collection
|
v
PySpark RDD
|
v
Data Cleaning
|
v
Feature Extraction
|
v
Anomaly Detection
|




**Transaction Data**

Each transaction may contain:

Transaction ID

User ID

Amount

Location

Timestamp

Merchant

Payment Method

Device ID

**RDD Processing**

Transaction records can be represented as:

(transactionID, userID, amount, location, timestamp)

RDD operations can clean and transform the data.

Example:

transactions.filter(lambda x: x.amount > 0)

Transactions can be grouped by user:

transactions.map(lambda x: (x.userID, x)).groupByKey()

reduceByKey() can be used to calculate transaction totals.

**Feature Extraction**

Important fraud detection features include:

- Transaction amount
- Transaction frequency
- Geographic distance
- Time of transaction
- Device changes
- Unusual merchant
- Previous transaction history

**Anomaly Detection**

A transaction is considered suspicious if it significantly differs from the user's normal behavior.

Example:

Normal:
User usually spends ₹500–₹2,000

New transaction:
₹1,50,000 from another country

↓

High anomaly score
↓
Potential fraud
Machine-learning algorithms such as clustering or anomaly-detection models can be applied to distributed data.

**Intelligent Agents**

**Transaction Monitoring Agent**
Monitors incoming transactions.

**User Behavior Agent**
Maintains the user's normal spending pattern.

**Anomaly Detection Agent**
Calculates anomaly scores.

**Risk Assessment Agent**
Assigns a fraud-risk score.

**Alert Agent**
Generates alerts when risk exceeds a threshold.

**Agent Collaboration**
Transaction Agent
↓
Behavior Agent
↓
Anomaly Agent
↓
Risk Agent
↓
Alert Agent

**Continuous Learning**
When investigators classify a transaction as fraud or legitimate, the result is used as feedback.
Transaction
↓
Prediction
↓
Fraud/Normal
↓
Human Feedback
↓

Model Update
This helps improve future detection accuracy.

**Scalability**

PySpark distributes transactions across cluster nodes. Therefore, millions of transactions can be processed in parallel.

The system also provides:

- Fault tolerance
- Parallel computation
- Distributed storage
- High throughput
- Horizontal scalability

**Conclusion**

The proposed framework combines PySpark RDD processing, anomaly detection, and intelligent agents to detect fraudulent financial transactions efficiently. Continuous feedback allows the system to improve its accuracy over time.

**4. Autonomous Supply Chain Optimization System**

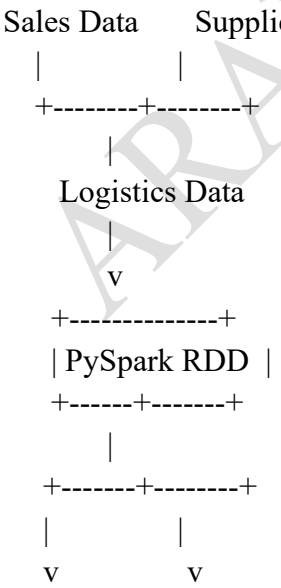
**Introduction**

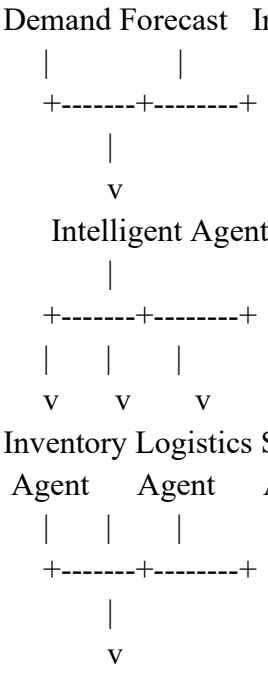
An Autonomous Supply Chain Optimization System uses distributed AI to optimize **inventory, logistics, and demand forecasting**. PySpark RDD transformations process large-scale supply-chain data, while multiple intelligent agents make decentralized decisions for cost minimization and efficiency improvement.

**Objectives**

- Optimize inventory levels.
- Predict product demand.
- Select efficient transportation routes.
- Minimize logistics costs.
- Avoid stock-outs and overstocking.
- Make decentralized supply-chain decisions.

**System Architecture**






**Data Sources**

The system processes:

- Historical sales
- Inventory records
- Supplier information
- Transportation costs
- Delivery times
- Customer orders
- Seasonal demand

**RDD Transformations**

Data can be represented as:

(productID, quantity, date, location)

RDD transformations include:

**map()**

Transforms raw records into useful structures.

**filter()**

Removes invalid or unnecessary records.

**reduceByKey()**

Calculates total product sales.

**groupByKey()**

Groups data by product or location.

**join()**

Combines inventory and sales information.

Example:

```
sales.map(lambda x: (x.productID, x.quantity))
    .reduceByKey(lambda a, b: a + b)
```

**Demand Forecasting**

Historical sales are analyzed to estimate future demand.

Historical Sales

↓

RDD Processing

↓

Pattern Analysis

↓

Demand Forecast

↓

Inventory Decision

If demand is expected to increase, the system can recommend increasing inventory.

**Intelligent Agents**

**Inventory Agent**

Monitors stock levels and recommends replenishment.

**Logistics Agent**

Selects efficient delivery routes.

**Supplier Agent**

Evaluates suppliers based on:

- Cost
- Quality
- Delivery time
- Reliability

**Demand Forecasting Agent**

Predicts future product demand.

**Optimization Agent**

Combines the outputs of other agents and selects the best decision.

**Decentralized Decision Making**

Agents can operate independently but share information.

For example:

Demand Agent → predicts high demand

↓

Inventory Agent → requests additional stock

↓

Supplier Agent → selects supplier

↓

Logistics Agent → selects delivery route

↓

Optimization Agent → final decision

**Cost Minimization**

The system can consider:

Total Cost =

Inventory Cost

+ Transportation Cost

+ Ordering Cost

+ Storage Cost

The optimal solution minimizes the overall cost while maintaining sufficient inventory.

**Scalability**

PySpark allows large supply-chain datasets to be processed across multiple machines.

This enables the system to handle:

- Multiple warehouses
- Thousands of products
- Large numbers of suppliers
- Millions of transactions

**Conclusion**

The autonomous supply-chain system combines distributed PySpark processing with intelligent agents. It enables organizations to forecast demand, optimize inventory, select efficient routes, and reduce operational costs.

**5. Personalized Learning Analytics Platform**

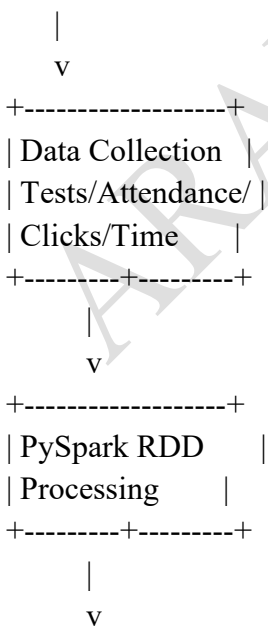
**Introduction**

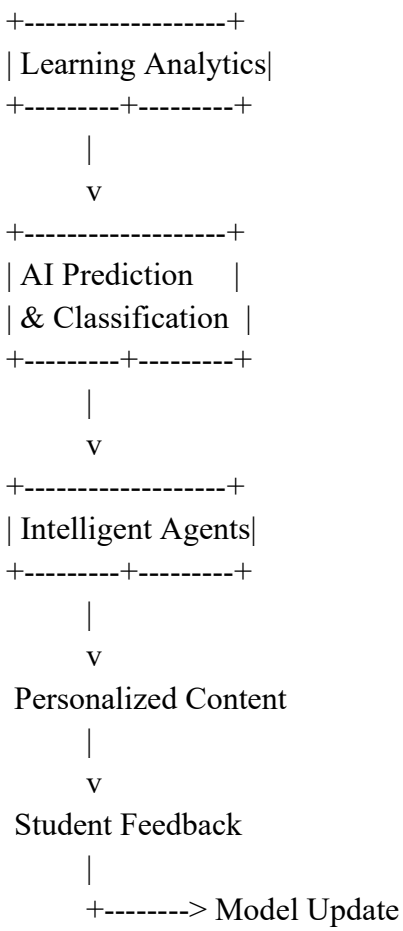
A Personalized Learning Analytics Platform is an AI-driven educational system that analyzes student learning behavior and performance using PySpark and RDDs. Intelligent agents provide adaptive learning, real-time feedback, and personalized content delivery for thousands of learners.

**Objectives**

- Analyze student performance.
- Identify strengths and weaknesses.
- Recommend personalized learning materials.
- Provide real-time feedback.
- Detect students requiring additional support.
- Scale the platform for thousands of students.

**System Architecture**






**Student Data**

The system can collect:

- Test scores
- Assignment marks
- Attendance
- Time spent studying
- Video watching time
- Quiz attempts
- Number of incorrect answers
- Learning progress

**RDD Processing**

Student records can be represented as:

(studentID, subject, score, studyTime)

RDD operations can process this data.

Example:

studentRDD.filter(lambda x: x.score < 50)

can identify students with low scores.

map() can transform records, while reduceByKey() can calculate average performance.

**Learning Analytics**

The system calculates:

- Average score
- Subject-wise performance

- Learning speed
- Weak topics
- Strong topics
- Engagement level

Example:
Mathematics → 85%
Programming → 72%
Cybersecurity → 45%

Weak Area → Cybersecurity
The system can recommend additional cybersecurity learning materials.

**Intelligent Agents**

**Student Monitoring Agent**

Tracks student activity and performance.

**Performance Analysis Agent**

Identifies strengths and weaknesses.

**Content Recommendation Agent**

Selects suitable learning materials.

**Feedback Agent**

Provides immediate feedback after tests and activities.

**Intervention Agent**

Identifies students who require additional assistance.

**Adaptive Learning**

The platform adapts content according to the student's performance.

Student completes quiz
↓
Performance analyzed
↓
Weak topic identified
↓
Difficulty adjusted
↓
Personalized content provided
↓
Student completes activity
↓
Performance updated

For a high-performing student, the system can provide advanced questions. For a struggling student, it can provide simpler explanations and additional practice.

**Real-Time Feedback**

Suppose a student answers five questions incorrectly on a particular topic.
The system can immediately:

1. Identify the weak concept.
2. Provide an explanation.

3. Recommend a tutorial.
4. Generate additional practice questions.
5. Monitor improvement.

**Scalability**

PySpark enables the platform to process learning data from thousands of students simultaneously.
RDD partitioning provides:

- Parallel processing
- Distributed computation
- Fault tolerance
- Efficient large-scale analytics

**Innovation**

An important innovation is the use of multiple intelligent agents instead of a single centralized decision-maker.
For example:
Performance Agent
+
Behavior Agent
+
Recommendation Agent
+
Feedback Agent
↓
Personalized Learning Plan

**Conclusion**

The Personalized Learning Analytics Platform combines PySpark RDDs, AI analytics, and intelligent agents to provide scalable and adaptive education. It continuously analyzes student behavior and performance and adjusts learning content according to individual needs.

**Overall Data Flow for the 5 Systems**

The common architecture can be represented as:

DATA SOURCES
↓
DATA COLLECTION
↓
PYSPARK / RDD
↓
DATA PREPROCESSING
↓
FEATURE EXTRACTION
↓
AI / ML MODEL
↓
INTELLIGENT AGENTS
↓

DECISION MAKING
↓
USER / SYSTEM
RESPONSE
↓
FEEDBACK
↓
MODEL UPDATE

**Important PySpark RDD Operations to Mention**

| Operation | Purpose |
|---|---|
| map() | Transform each data element |
| filter() | Select required records |
| flatMap() | Generate multiple outputs from one record |
| groupByKey() | Group values by key |
| reduceByKey() | Aggregate values efficiently |
| join() | Combine two RDDs |
| sortByKey() | Sort key-value data |
| count() | Count records |
| collect() | Retrieve RDD results |
| reduce() | Aggregate complete RDD |